

VFSForGit

.NET Framework 4.7.1 → .NET 10 NativeAOT

Incremental migration path from legacy .NET Framework to native-compiled binaries — with benchmark proof at every step.

Current State

VFSForGit ships as .NET Framework 4.7.1 with:

- Newtonsoft.Json for serialization
- CommandLineParser for CLI
- WMI/COM for disk info
- C++/CLI mixed-mode ProjFS wrapper
- ~15 managed executables + native hooks

Problems:

- JIT startup penalty on every `gvfs` invocation
- No NativeAOT (blocked by C++/CLI + Newtonsoft.Json)
- Large deployment (full .NET Framework required)

Migration Strategy

Six incremental phases — each independently reviewable and testable:

Phase	Change	Risk
1. TFM Retarget	net471 → net10.0-windows	Low
2. Package Updates	Update NuGet deps for .NET 10	Low
3. API Migrations	ACL, NamedPipe, HttpUtility	Medium
4. Newtonsoft → STJ	System.Text.Json + source generators	Medium
5. Other Modernizations	System.CommandLine, WMI→P/Invoke	Low
6. NativeAOT	C++/CLI→C# ProjFS, PublishAot=true	High value

Each phase has a commit, passes all tests, and can be reviewed independently.

Phase 1-3: Foundation

TFM Retarget (1 commit)

- All 18 .csproj files: `net471` → `net10.0-windows10.0.17763.0`
- Directory.Build.props: centralized TFM + RID

Package Updates (1 commit)

- NuGet packages updated to .NET 10 compatible versions
- ESENT, NUnitLite, SSH.NET, etc.

API Migrations (several commits)

- `DirectorySecurity` → `System.IO.FileSystemAclExtensions`
- `NamedPipeServerStream` constructor changes
- `HttpUtility.UrlEncode` → `System.Net.WebUtility`

Phase 4: Newtonsoft.Json → System.Text.Json

Why: Newtonsoft.Json uses reflection heavily — incompatible with AOT trimming.

Approach

- Created `GVFSJsonOptions.cs` — shared options with source-generated contexts
- Created `GVFSJsonContext.cs` — `[JsonSerializable]` for all serialized types
- `VersionConverter` — handles `System.Version` as both string and JSON object
- `EventMetadataConverter` — AOT-safe `Dictionary<string, object>` serialization

Key Lesson: STJ Source Generator Naming Collisions

Multiple nested types with the same short name (e.g., six different `Response` classes) cause the source generator to **silently drop all but one**.

Fix: `TypeInfoPropertyName` to disambiguate.

Phase 5: Other Modernizations

System.CommandLine (AOT branch)

- Replaced CommandLineParser 2.6.0 with System.CommandLine 2.0.3 (stable)
- 15 verb commands with backward-compatible `version` subcommand
- Uses `SetAction` + `ParseResult` pattern (2.0.x stable API)
- `ExecuteVerb()` wrapper for `VerbAbortedException` handling

WMI → P/Invoke (AOT branch)

- `WindowsPhysicalDiskInfo.cs` rewritten from WMI/COM to `DeviceIoControl`
- `IOCTL_STORAGE_QUERY_PROPERTY` + `IOCTL_DISK_GET_DRIVE_GEOMETRY`
- Eliminates COM interop dependency

Phase 6: NativeAOT

C++/CLI → Pure C# ProjFS

- 5 files, ~1,700 lines of pure C# P/Invoke
- Same `Microsoft.Windows.ProjFS` namespace — no consumer code changes
- `PRJ_PLACEHOLDER_INFO` struct: 344 bytes (including `VariableData[1]`)
- 16/16 ProjFS tests pass

PublishAot=true

- All 5 managed executables compile to native binaries
- GVFS.exe: 14.5 MB native binary (no JIT, no .NET runtime)
- Self-contained — zero external runtime dependencies

Benchmark Results

NativeAOT .NET 10 vs Production .NET Framework 4.7.1:

Benchmark	.NET FW	NativeAOT	Change
Startup	53.5 ms	15.3 ms	-71% (3.5× faster)
Status	341 ms	179 ms	-47% (1.9× faster)
Mount	4,603 ms	4,148 ms	-10%
Git Status	192 ms	139 ms	-27%
Git Log	140 ms	119 ms	-15%
Dir Enum	187 ms	186 ms	~same
File Read	8.3 ms	1.8 ms	-78%
Clone	5,234 ms	5,287 ms	~same (network)

Zero regressions. Dir enumeration now correct (864 files, 0 errors).

Deployment Comparison

Aspect	.NET Framework 4.7.1	NativeAOT .NET 10
Runtime required	.NET Framework 4.7.1	None (self-contained)
VC++ Redist required	Yes (ljwghost.dll)	No
Startup time	~54 ms (JIT)	~15 ms (native)
Binary type	IL + JIT	Native x64
GVFS.exe size	~200 KB IL + runtime	14.5 MB native
Total layout	~50 MB (with runtime)	~57 MB (all-in-one)

The ProjFS Connection

VFSForGit currently **vendors** ProjFS P/Invoke code.

The ProjFS-Managed-API repo has the **upstream** pure C# implementation.

Migration path:

1.  ProjFS-Managed-API: Pure C# replacement (16/16 tests pass)
2.  VFSForGit: Uses vendored copy (benchmarked, functional)
3.  ProjFS-Managed-API: Publish as NuGet package
4.  VFSForGit: Switch from vendored code to upstream NuGet

This keeps both repos in sync and lets other ProjFS consumers benefit.

Ask

For the VFSForGit team:

1. Review the phased PRs (6 phases, each independently testable)
2. Validate on your CI/CD pipeline
3. Ship the NativeAOT build to production

For the ProjFS team:

1. Accept the pure C# replacement PR
2. Publish updated NuGet package (net8.0+)
3. Deprecate C++/CLI package for new consumers

Benefits to both:

- **Faster** — 3.5× startup, 2× status, 78% faster reads